

Learned Bots for Jokerless

The network we chose, how we train it, and why it was the right fit

Hold'em · Canasta · Žolíky — status as of 30 September 2026

1. Summary

We replaced hand-written opponents with bots that **learn by playing**. All three games use one small neural-network design: a **candidate-scoring actor-critic MLP**. It looks at the position from one seat's point of view, scores every legal move, and plays the best one. Training uses **PPO self-play against a league** of rule-based bots, practice opponents that copy human habits, and the model's own earlier versions. The trained weights run in **plain Go** inside the game server, with no new runtime, and they can be switched on per game from the admin console.

- **Žolíky (v2)**: beats the rule-based Hard bot by +117 penalty points per match at the 4-player, 35-minimum table and by +160 one-on-one. It also beat a human tester decisively. It is ready for the Hard switch.
- **Hold'em**: +21 big blinds per match against Hard heads-up, but it overplays all-ins against a solid caller. It needs tighter practice opponents.
- **Canasta**: +7,003 points per match against Hard, but it was vulnerable to fast closers (Claude, playing through the MCP bridge, beat it by 850 in a test deal). A retrain against a closer bot was running when this was written.

2. The problem the network has to solve

A card-game bot here works under constraints that rule out many textbook designs:

- **Hidden information**. No seat may see another seat's cards or the stock. Every input the network gets comes from one seat's legal view, and each game has a test that changes hidden cards and checks the input does not change.
- **A different number of legal moves every turn**. Hold'em has about 7. Canasta averages 9 and peaks near 60. Žolíky ranges from 1 to 64, and a single move can be a whole multi-step opening.
- **The engine is the only rule authority**. The bot may only choose among moves the engine offers. It must never decide for itself that a move is legal.
- **It runs inside the Go server**. No cgo, no ONNX runtime in the image, deterministic under the match seed, and fast enough that a bot move takes well under the server's 5-second budget. In practice it takes roughly 0.3–2 ms.
- **One design for many games**. Adding a game should mean writing an adapter, not a new architecture.

3. The network

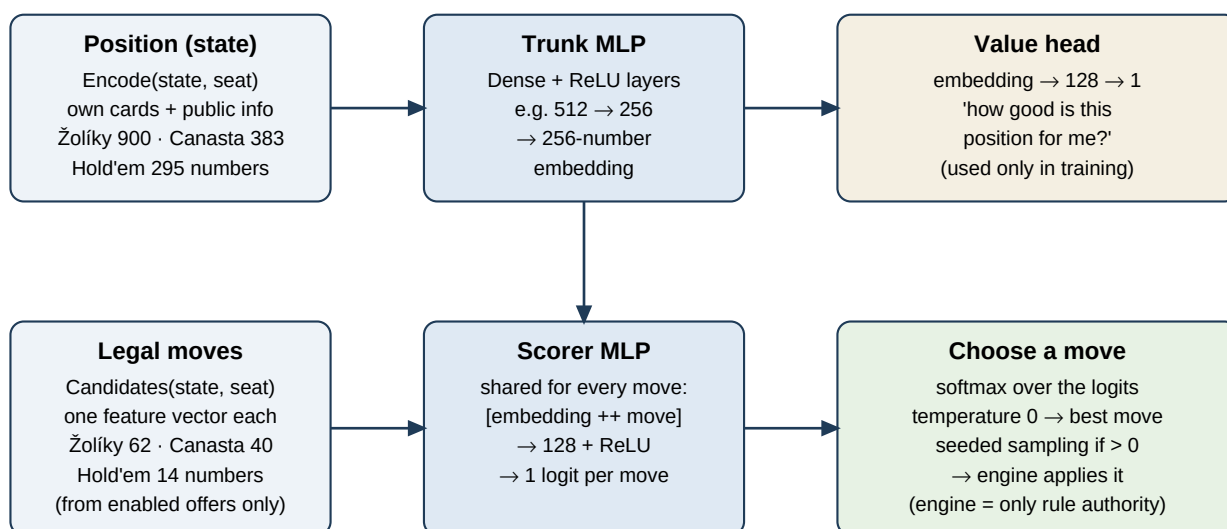


Figure 1. The candidate-scoring actor-critic network (identical shape for all three games).

Trunk. A fully-connected network (ReLU after every layer) turns the position into a 256-number embedding. The position covers the seat's own hand, the table, the discard pile, public memory of what each opponent has taken, how far the seat is from going down, the scores, the rules in force, and so on.

Scorer. For each legal move, the embedding is joined with that move's own features. These include the kind of move, the cards involved, wild cards spent, points laid, whether it goes out, whether a discard would feed the next player, and the hand left afterwards. A small network turns each pair into one score. The *same* scorer is used for every move, so the network handles 1 move or 60 moves without change.

Value head. It estimates how good the position is for the seat. It is used only during training to reduce noise, and the server ignores it.

Choosing. A softmax over the scores gives move probabilities. Hard seats take the highest-scoring move (temperature 0). Sampling, when used, is seeded from what the seat can see, so a game replays exactly.

Game	Position inputs	Move features	Trunk	Scorer / Value	Parameters	File size
Hold'em	295	14	256 → 256	128 → 1 / 128 → 1	≈ 209 k	0.84 MB
Canasta	383	40	256 → 256	128 → 1 / 128 → 1	≈ 236 k	0.94 MB
Žolíky v2	900	62	512 → 256	128 → 1 / 128 → 1	≈ 667 k	2.67 MB

The weights are written once by PyTorch in a small self-describing format (ZLNET1), and the server reads them with about 150 lines of Go. A parity test checks that Go and PyTorch produce the same scores to within 0.00001.

4. Why this network

We did not run a contest between architectures. We chose this design because it is the simplest one that meets every constraint in section 2, and the results in section 6 show it is enough. Each alternative we considered fails at least one constraint:

Alternative	Why we did not choose it
Fixed output layer (one neuron per possible move)	Rummy moves are combinations: which cards, which meld, which multi-step opening. There is no fixed list of moves to give one neuron each. Scoring candidates works for any number of moves and keeps legality with the engine.
Transformer / attention over cards, or an LSTM over history	More expressive, but slower and harder to run in plain Go, and harder to check against PyTorch. The hand-built encoder already gives the network the public history (what each opponent took and melded) as summary numbers, so a plain MLP sees what it needs. Worth revisiting if the MLP plateaus.
Game-tree search (alpha-beta, MCTS)	Works well for perfect-information games, and the repo uses alpha-beta for trick-taking (Mariáš's double-dummy solver). In rummy the hidden stock and opponents' hands, plus dozens of moves per turn over dozens of turns, make the tree explode. A learned policy is the practical base, and a shallow search on top of it remains possible later.
Counterfactual regret minimisation (CFR, as in poker solvers)	Very strong for heads-up Hold'em, but it needs abstractions specific to each game and does not carry over to rummy games. One method for all games mattered more here.
Learning only from human games (imitation)	Too little data for now. Self-play produces millions of decisions an hour. The exporter is built and human games will be used later, as evaluation and as an imitation opponent.
An LLM (Claude) as the in-game player	Seconds and a fee per move, and not deterministic enough to replay. Claude is used instead to design practice bots (the 'closer'), to review the model's decisions through the MCP bridge, and to play test games, which is where it adds the most.

What this design gives us in practice:

- **Legal by construction:** it can only ever pick a move the engine offered. There were 0 illegal moves across tens of millions of training decisions and every evaluation.
- **Small and fast:** under 3 MB per game and about 0.3–2 ms per move. One copy is loaded per server and shared by every table and seat; moving to the next player only changes the arguments.
- **Fair:** no-peek tests show the input depends only on that seat's legal view.
- **Reusable:** the same trainer, bench, environment, exporter and admin switch serve all three games. A new game needs only its Encode / Candidates / Reward adapter.

5. Training strategy

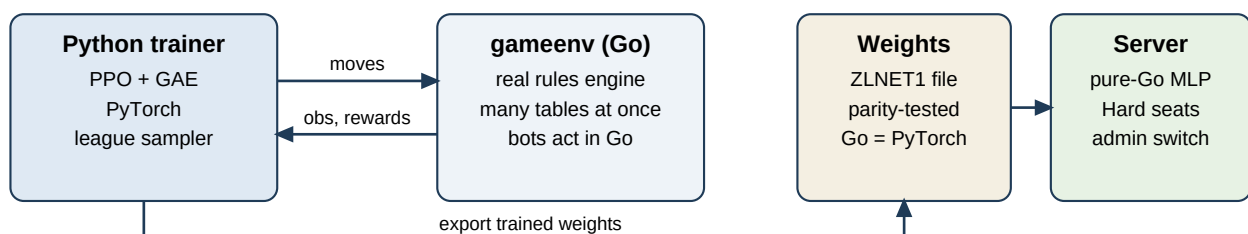


Figure 2. Training and shipping pipeline. Python never re-implements a game rule.

Algorithm: PPO with GAE. Proximal Policy Optimisation is the standard, stable choice for self-play with many parallel games. It copes with a different number of moves per decision, and its clipping and KL limit stop one update from wrecking the policy. Rewards follow the game's own scoring: chips per hand in Hold'em, and

per-deal point difference in Canasta and Žolíky (partners share it).

A league, not just self-play. Pure self-play tends to find tricks that only fool copies of itself. Each training table draws its opponents from a mix:

- **Rule-based bots** (Easy, Medium, Hard): a steady, known baseline.
- **Practice styles that copy human habits:** maniac, rock, calling station and river-bluffer for Hold'em; 'closer' for Žolíky and Canasta, which goes down early and out fast. Each was added after a human or Claude test exposed a weakness.
- **Frozen earlier versions of the model**, so it keeps improving against itself.
- **Self-play and mixed partners:** in Canasta, half the time the model's partner is rule-based Hard, because in real games its partner may be a person or the old bot.

Measurement. Every comparison uses a duplicate bench. Each unseen seed (1,000,000 and up, never used in training) is played twice with the seats swapped, so card luck cancels out. Results are reported with standard errors. A model that cannot fit the current game code is refused.

Improving by iteration. Each round followed the same loop: train, benchmark, then play it (the owner in the browser, Claude through the MCP bridge), find what the benchmark missed, encode the fix (new inputs, a new practice bot, removing an over-cautious move filter), and retrain. Later rounds start from the previous weights (*--init*), and when inputs are added they start at zero (*--widen*), so nothing learned is lost.

Setting	Hold'em	Canasta (v2)	Žolíky (v2)
Learning rate (floor)	3e-4 (1%)	3e-4 (20%)	4e-4 (20%)
Discount γ / GAE λ	1.0 / 0.95	0.999 / 0.95	0.999 / 0.95
Entropy bonus	0.01	0.012	0.01
Decisions per update	16,384	16,384	16,384
Table sizes	2–6 seats	65% 4-seat, 35% 2-seat	70% 4-seat 35-min, 30% 2-seat
League (heuristic / checkpoints / self)	40 / 40 / 20 %	55 / 25 / 20 %	55 / 25 / 20 %
Key practice opponent	maniac, station	closer $\times 1.5$	closer $\times 1.5$

6. Results

Duplicate bench, held-out seeds, temperature 0, $\pm$ one standard error. Positive means the model does better.

Žolíky (penalty points per match; v2 model)

Table	vs Hard	vs closer	For scale: Hard vs Medium
4 players, 35 minimum, 2 models vs 2 Hard	+117 $\pm$ 2	+88 $\pm$ 2	+39 $\pm$ 3
1 model vs 3 Hard / 3 closers	+109 $\pm$ 2	+104 $\pm$ 2	—
2 players, classic	+160 $\pm$ 3	+173 $\pm$ 2	+54 $\pm$ 4

Owner's game at the 4-player 35-minimum table: the three model bots finished on 25, 34 and 134 points, against the human's 270, over three deals. An earlier model had lost a deal 171–0 by never finding a clean run; the v2 inputs and the closer opponent were the fix.

Hold'em (big blinds per match, heads-up unless noted)

Opponent	Model	Hard on the same seeds
Hard	+21.4 $\pm$ 1.4	—
Maniac / Rock	+34.9 / +22.3	+23.1 / +7.0
Calling station	+57.0 $\pm$ 1.6	+61.4
Hard, 6 seats	+13.4 $\pm$ 1.1	—

In a test game through the MCP bridge it limped, re-raised and then shoved all-in on the flop (92% confident) with a weaker hand than top pair, and lost. It needs tighter, calling opponents in its league.

Canasta (points per match)

Matchup	2 players	4 players (partnerships)
Model vs Hard	+7,003 $\pm$ 74	+3,084 $\pm$ 78
Model vs closer	+5,789 $\pm$ 82	+1,199 $\pm$ 84
Model + Hard partner vs two closers	—	+612 $\pm$ 84

It wins by building canastas on long deals and rarely closes, which a fast closer punishes. A Claude test deal was lost by 850 points. The retrain against the closer (65% 4-player, half the time with Hard partners) was nearly finished when this was written. Its training trend roughly doubled the reward per deal against 4-player closers. Its targets: +2,500 against the closer at 4 players, +1,500 with a Hard partner, and keeping its edge over Hard.

7. Lessons, limits and next steps

- **Benchmarks against rule-based bots overstate strength** when the bots share a weakness (slow closing). Human and Claude test games found every important flaw. We recommend making a few human or Claude test games part of the release gate for each model.
- **Most gains came from the inputs and the practice opponents, not from network size.** The v2 Žolíky inputs (distance to going down, what opponents collect) and the closer helped more than longer training.
- **Rules can look like strategy.** A suspected 'missing pickup' turned out to be a rule: before going down, a pile card must be used that turn. Understanding the rules engine came first.
- **Limits:** no look-ahead search, a fixed hand-built encoder, and training mostly against bots. Real human games from the match history (exporter ready) are the next data source.
- **Shipping:** the models are embedded in the server and switched per game from the admin console. They are off by default, take effect on the next move, the setting persists, and every change is logged. Žolíky is recommended for 'on'. Canasta should be switched on after its retrain passes, and Hold'em after a tighter-opponent retrain.